



AMRITA SCHOOL OF ENGINEERING, BENGALURU

DEPARTMENT OF ELECTRONICS & COMMUNICATION
ENGINEERING

B. TECH. II SEMESTER(ECE/EAC) AY 2023-2024
23ECE184 INTRODUCTION TO INTERNET OF THING
LAB
SYSTEM DESIGN REPORT
MAY 2025

Submitted by

BL.EN.U4EAC24009
BL.EN.U4EAC24010
BL.EN.U4EAC24011

B. AKSHAY SARADHI
B. ADITYA
BALAJI RAYUDU

BATCH NUMBER: 05

Faculty Examiner Signature with Date

TITLE: - HEALTH MONITORING SYSTEM

OBJECTIVE: - To create a wearable device using ESP32, MAX30102, display, and BAM400 to continuously monitor and log real-time health data (heart rate and SpO2)

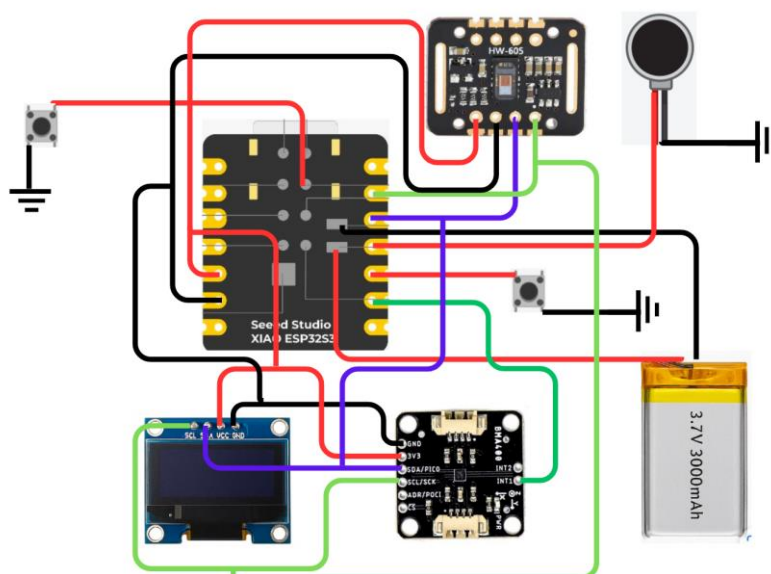
COMPONENTS REQUIRED ALONG WITH THEIR SPECIFICATIONS: -

Sl. No	Name of the Component	Specification	Quantity
1.	ESP 32	S3	1
2.	DISPLAY	OLED	1
3.	BMA400		1
4.	MAX30102		1
5.	WATCH 3D CASE	3D PRINTED	1
6.	BATTERY	LI-PO 300Mah	1
7.	BUTTON MOTOR	5mm	1

MOTIVATION/SOCIATAL/INDUSTRIAL IMPACT: -

This wearable health monitoring device is designed to address the growing demand for continuous, real-time health tracking. By measuring heart rate, SpO₂, and physical activity, it empowers individuals to manage their well-being proactively. It offers significant societal benefits by aiding early detection of health issues, supporting elderly care, and providing health access in remote areas. Industrially, it contributes to the expansion of digital healthcare, remote patient monitoring, and fitness tech, reducing the burden on traditional medical systems while enabling scalable, data-driven healthcare solutions.

BLOCK DIAGRAM/SYSTEM DESIGN: -



CODE (ESP32 AND MIT APP CODE): -

MIT APP CODE: -

```
when Screen1 ▾ .Initialize
do
  call WebView1 ▾ .GoToUrl
    url "file:///android_asset/iot.html"
```

```
when WebView1 ▾ .WebViewStringChange
  value
do
  if WebView1 ▾ .WebViewString = "a"
  then
    open another screen screenName ble ▾
  close screen
```

```
initialize global service to "12345678-1234-5678-1234-56789abcdef0"
```

```
initialize global char1 to "abcdef12-3456-7890-abcd-ef1234567890"
```

```
when ble ▾ .Initialize
do
  call ble ▾ .AskForPermission
    permissionName "BLUETOOTH_CONNECT"
```

```
when ble ▾ .PermissionGranted
  permissionName
do
  if get permissionName = "BLUETOOTH_CONNECT"
  then
    call ble ▾ .AskForPermission
      permissionName "BLUETOOTH_SCAN"
```

```

when Button1 .Click
do call BluetoothLE1 .StartScanning

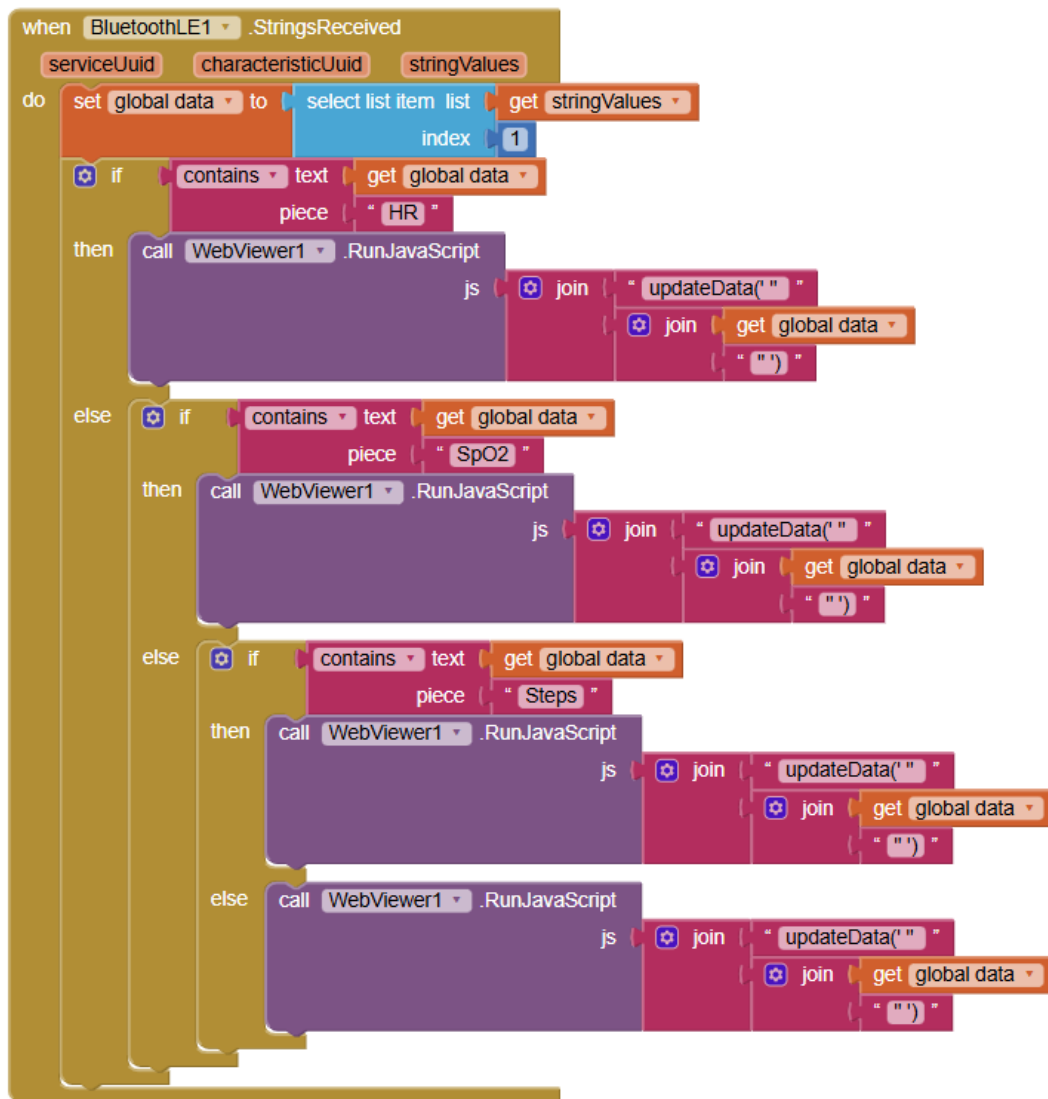
when BluetoothLE1 .DeviceFound
do set ListPicker1 .ElementsFromString to BluetoothLE1 . DeviceList

when BluetoothLE1 .Connected
do
  set WebView1 . Visible to true
  set Label5 . Visible to false
  set Button1 . Visible to false
  set ListPicker1 . Visible to false
  call BluetoothLE1 .RegisterForStrings
    serviceUuid get global service
    characteristicUuid get global char1
    utf16 false
  call WebView1 .GoToUrl
    url "file:///android_asset/health_details.html"

when ListPicker1 .AfterPicking
do
  if ListPicker1 . SelectionIndex ≥ 0
  then
    call BluetoothLE1 .StopScanning
    call BluetoothLE1 .Connect
      index ListPicker1 . SelectionIndex

initialize global data to ""

```



ESP 32 CODE:-

```

#include <Arduino.h>
#include <Wire.h>
#include "SparkFun_BMA400_Arduino_Library.h"
#include "MAX30105.h"
#include "spo2_algorithm.h"
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <NimBLEDevice.h>

```

```

#define SCREEN_WIDTH 128

```

```

#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define BUTTON_PIN 3
#define HAPTIC_PIN 4
#define DEBOUNCE_DELAY 50
#define LONG_PRESS_THRESHOLD 2500
#define AUTO_OFF_MS 60000
#define MIN_IR_THRESHOLD 50000
#define NO_FINGER_CONFIRM_MS 100

#define BATTERY_PIN A0
#define R_TOP 47000.0
#define R_BOTTOM 100000.0
#define V_REF 3.3
#define ADC_MAX 4095.0

#define SENSOR_SERVICE_UUID "12345678-1234-5678-1234-56789abcdef0"
#define SENSOR_CHAR_UUID "abcdef12-3456-7890-abcd-ef1234567890"

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);
BMA400 accelerometer;
uint8_t i2cAddress = BMA400_I2C_ADDRESS_DEFAULT;
const int interruptPin = 2;
volatile bool interruptOccurred = false;
uint32_t bmaStepCount = 0;
uint8_t bmaActivityType = 0;

MAX30105 particleSensor;
bool maxPresent = true;
const byte LED_BRIGHTNESS = 0x1F;
const int SAMPLE_RATE = 300;
const int DATA_BUFFER_SIZE = 100;
const int PULSE_WIDTH = 411;
const int ADC_RANGE = 4096;
uint32_t irBuffer[DATA_BUFFER_SIZE];
uint32_t redBuffer[DATA_BUFFER_SIZE];
int bufferIndex = 0;
int32_t spo2;
int8_t validSPO2;
float bpm = 0;
unsigned long lastBeatTime = 0;
const int HR_FILTER_SIZE = 10;
float beatIntervals[HR_FILTER_SIZE] = {0};
int beatIntervalIndex = 0;

enum DeviceState {
    OFF_STATE,
    STEPS_STATE,
    HR_STATE,

```

```

    LONG_ON_STATE
};
DeviceState deviceState = OFF_STATE;
bool longPressActive = false;
bool buttonCurrentlyPressed = false;
unsigned long buttonPressTime = 0;
unsigned long lastActivityTime = 0;
static bool prevButtonState = HIGH;
static unsigned long lastChangeTime = 0;

static unsigned long noFingerStartTime = 0;
static bool noFingerConfirmed = false;

NimBLEServer* pBLEServer = nullptr;
NimBLEService* pBLEService = nullptr;
NimBLECharacteristic* pSensorCharacteristic = nullptr;

void setupBLE() {
    NimBLEDevice::init("SensorDevice");
    pBLEServer = NimBLEDevice::createServer();
    pBLEService = pBLEServer->createService(SENSOR_SERVICE_UUID);
    pSensorCharacteristic = pBLEService->createCharacteristic(
        SENSOR_CHAR_UUID,
        NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::NOTIFY
    );
    pSensorCharacteristic->setValue("No Data");
    pBLEService->start();
    NimBLEAdvertising* pAdvertising = NimBLEDevice::getAdvertising();
    pAdvertising->setName("SensorDevice");
    pAdvertising->addServiceUUID(SENSOR_SERVICE_UUID);
    pAdvertising->start();
}

void updateBLE() {
    if (!pBLEServer || pBLEServer->getConnectedCount() == 0) return;

    static uint8_t bleState = 0;
    static unsigned long lastBLETime = 0;
    const unsigned long gapTime = 1000;

    if (millis() - lastBLETime < gapTime) return;
    lastBLETime = millis();

    String dataToSend;
    switch (bleState) {
        case 0: dataToSend = "HR:" + String(bpm, 1); break;
        case 1: dataToSend = "SpO2:" + String(spo2); break;
        case 2: dataToSend = "Steps:" + String(bmaStepCount); break;
        case 3: {
            String activity;

```

```

switch (bmaActivityType) {
    case BMA400_RUN_ACT: activity = "Running"; break;
    case BMA400_WALK_ACT: activity = "Walking"; break;
    case BMA400_STILL_ACT: activity = "Standing"; break;
    default: activity = "Unknown"; break;
}
dataToSend = "Activity:" + activity;
break;
}
}

pSensorCharacteristic->setValue(dataToSend.c_str());
pSensorCharacteristic->notify();
bleState = (bleState + 1) % 4;
}

void setupHaptic() {
    pinMode(HAPTIC_PIN, OUTPUT);
    digitalWrite(HAPTIC_PIN, LOW);
}

void buzzHaptic(unsigned long durationMs) {
    digitalWrite(HAPTIC_PIN, HIGH);
    delay(durationMs);
    digitalWrite(HAPTIC_PIN, LOW);
}

int getBatteryPercentage() {
    static int lastPct = -1;
    float raw = analogRead(BATTERY_PIN);
    float v_adc = (raw / ADC_MAX) * V_REF;
    float v_bat = v_adc * (R_TOP + R_BOTTOM) / R_BOTTOM;
    v_bat = constrain(v_bat, 3.0, 4.2);
    int pct = map(v_bat * 100, 300, 420, 0, 100);
    if (lastPct < 0 || abs(pct - lastPct) >= 3) lastPct = pct;
    return lastPct;
}

void drawBatteryIcon() {
    int pct = getBatteryPercentage();
    int x = SCREEN_WIDTH - 28;
    int y = 0;
    int w = 20;
    int h = 8;
    int level = map(pct, 0, 100, 0, w - 4);

    display.drawRect(x, y, w, h, SSD1306_WHITE);
    display.drawRect(x + w, y + 2, 2, 4, SSD1306_WHITE);
    display.fillRect(x + 2, y + 2, level, h - 4, SSD1306_WHITE);
}

```

```

display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(x - 18, y);
display.print(pct);
display.print("%");
}

void setup() {
  Serial.begin(115200);
  Wire.begin(5, 6);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  setupHaptic();

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    while (1);
  }
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("Combined Sensor Init");
  display.display();
  delay(1000);

  while (accelerometer.beginI2C(i2cAddress) != BMA400_OK) {
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("BMA400 not connected!");
    display.display();
    delay(1000);
  }

  bma400_step_int_conf config = { .int_chan = BMA400_INT_CHANNEL_1 };
  accelerometer.setStepCounterInterrupt(&config);
  accelerometer.setInterruptPinMode(BMA400_INT_CHANNEL_1,
BMA400_INT_PUSH_PULL_ACTIVE_1);
  accelerometer.enableInterrupt(BMA400_STEP_COUNTER_INT_EN, true);
  attachInterrupt(digitalPinToInterrupt(interruptPin), bma400InterruptHandler,
RISING);

  if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
    maxPresent = false;
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("MAX30105 not");
    display.println("connected!");
    display.display();
    delay(2000);
  } else {

```

```

    particleSensor.setup(LED_BRIGHTNESS, 4, 2, SAMPLE_RATE,
PULSE_WIDTH, ADC_RANGE);
    particleSensor.enableDIETEMPRDY();
    delay(3000);
}

setupBLE();
}

void loop() {
    handleButtonPress();

    if (!longPressActive && (deviceState == STEPS_STATE || deviceState ==
HR_STATE)) {
        if (millis() - lastActivityTime >= AUTO_OFF_MS) deviceState = OFF_STATE;
    }

    if (deviceState == OFF_STATE) {
        display.ssd1306_command(SSD1306_DISPLAYOFF);
        return;
    } else {
        display.ssd1306_command(SSD1306_DISPLAYON);
    }

    if (interruptOccurred) {
        interruptOccurred = false;
        uint16_t interruptStatus = 0;
        accelerometer.getInterruptStatus(&interruptStatus);
        if (interruptStatus & BMA400_ASSERTED_STEP_INT) {
            accelerometer.getStepCount(&bmaStepCount, &bmaActivityType);
        }
    }

    if (maxPresent && (deviceState == HR_STATE || deviceState ==
LONG_ON_STATE)) {
        static unsigned long lastSample = millis();
        if (millis() - lastSample >= 10) {
            lastSample = millis();
            float irValue = particleSensor.getIR();
            float redValue = particleSensor.getRed();
            irBuffer[bufferIndex] = irValue;
            redBuffer[bufferIndex] = redValue;
            detectHeartBeat(irValue);
            bufferIndex++;
            if (bufferIndex >= DATA_BUFFER_SIZE) {
                processSpO2();
                bufferIndex = 0;
            }
        }
    }
}

```

```

    updateDisplay();
    updateBLE();
}

void handleButtonPress() {
    bool rawState = digitalRead(BUTTON_PIN);
    if (rawState != prevButtonState) lastChangeTime = millis();
    bool stableState = (millis() - lastChangeTime < DEBOUNCE_DELAY) ?
prevButtonState : rawState;
    prevButtonState = rawState;

    if (!buttonCurrentlyPressed && stableState == LOW) {
        buttonPressTime = millis();
        buttonCurrentlyPressed = true;
    }

    if (buttonCurrentlyPressed && stableState == LOW && !longPressActive &&
        millis() - buttonPressTime >= LONG_PRESS_THRESHOLD) {
        longPressActive = true;
        buzzHaptic(250);
        deviceState = LONG_ON_STATE;
        lastActivityTime = millis();
    }

    if (buttonCurrentlyPressed && stableState == HIGH) {
        if (millis() - buttonPressTime < LONG_PRESS_THRESHOLD) {
            buzzHaptic(150);
            if (deviceState == OFF_STATE) deviceState = STEPS_STATE;
            else if (deviceState == STEPS_STATE) deviceState = HR_STATE;
            else if (deviceState == HR_STATE) deviceState = OFF_STATE;
            else if (deviceState == LONG_ON_STATE) {
                deviceState = OFF_STATE;
                longPressActive = false;
            }
        }
        lastActivityTime = millis();
    }
    buttonCurrentlyPressed = false;
}

void detectHeartBeat(float irValue) {
    if (irValue < MIN_IR_THRESHOLD) return;
    static float threshold = 0;
    static bool pulseDetected = false;
    static float irHistory[DATA_BUFFER_SIZE] = {0};
    static int historyIndex = 0;

    irHistory[historyIndex] = irValue;
    historyIndex = (historyIndex + 1) % DATA_BUFFER_SIZE;
}

```

```

float mean = 0, sumSq = 0;
for (int i = 0; i < DATA_BUFFER_SIZE; i++) mean += irHistory[i];
mean /= DATA_BUFFER_SIZE;
for (int i = 0; i < DATA_BUFFER_SIZE; i++) sumSq += pow(irHistory[i] - mean,
2);
threshold = mean + sqrt(sumSq / DATA_BUFFER_SIZE);

if (!pulseDetected && irValue > threshold) {
    pulseDetected = true;
    unsigned long currentTime = millis();
    unsigned long beatDuration = currentTime - lastBeatTime;
    if (beatDuration > 300 && beatDuration < 2000) {
        beatIntervals[beatIntervalIndex] = 60000.0 / beatDuration;
        beatIntervalIndex = (beatIntervalIndex + 1) % HR_FILTER_SIZE;
        float total = 0; int count = 0;
        for (int i = 0; i < HR_FILTER_SIZE; i++) {
            if (beatIntervals[i] > 40 && beatIntervals[i] < 180) {
                total += beatIntervals[i]; count++;
            }
        }
        bpm = (count > 0) ? (total / count) : 0;
    }
    lastBeatTime = currentTime;
}
if (pulseDetected && irValue < threshold) pulseDetected = false;
}

void processSpO2() {
    if (irBuffer[0] < MIN_IR_THRESHOLD) return;
    int32_t heartRate; int8_t validHeartRate;
    maxim_heart_rate_and_oxygen_saturation(
        irBuffer, DATA_BUFFER_SIZE, redBuffer,
        &spo2, &validSPO2, &heartRate, &validHeartRate
    );
}

void updateDisplay() {
    display.clearDisplay();

    if (!maxPresent) {
        display.setCursor(0, 0);
        display.print("HR sensor X");
    }

    uint32_t currentIR = (bufferIndex > 0) ? irBuffer[bufferIndex - 1] : 0;
    display.setCursor(0, 8);
    if (currentIR < MIN_IR_THRESHOLD) {
        if (noFingerStartTime == 0) {
            noFingerStartTime = millis();

```

```

        noFingerConfirmed = false;
    } else if (millis() - noFingerStartTime >= NO_FINGER_CONFIRM_MS) {
        noFingerConfirmed = true;
    }
    } else {
        noFingerStartTime = 0;
        noFingerConfirmed = false;
    }

    if (noFingerConfirmed) {
        display.print("No finger");
    } else {
        display.print("SpO2: ");
        if (validSPO2) display.print(spo2);
        else display.print("---");
        display.print("%");
        display.setCursor(0, 16);
        display.print("BPM: ");
        if (bpm > 40 && bpm < 180) display.print(bpm);
        else display.print("---");
    }

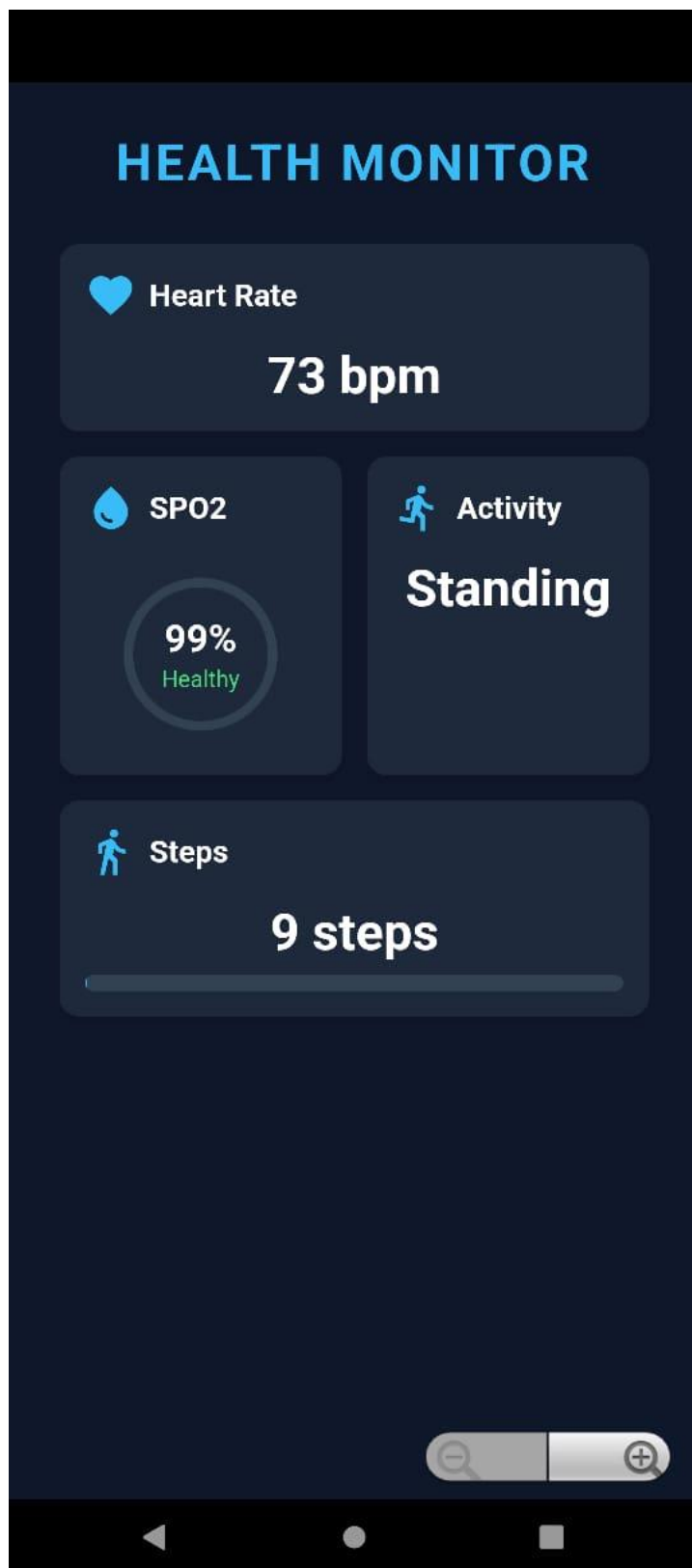
    display.setCursor(0, 32);
    display.print("Steps: ");
    display.print(bmaStepCount);

    display.setCursor(0, 42);
    display.print("Activity: ");
    switch (bmaActivityType) {
        case BMA400_RUN_ACT: display.print("Running"); break;
        case BMA400_WALK_ACT: display.print("Walking"); break;
        case BMA400_STILL_ACT: display.print("Standing"); break;
        default: display.print("Unknown"); break;
    }

    drawBatteryIcon();
    display.display();
}

```

SCREENSHOT OF ALL THE WORKING CIRCUITS WITH OUTPUT: -





OBSERVATION TABLE: -

Heart Rate (BPM)	SpO ₂ (%)	Steps	Activity	Battery (%)	Finger Detected	BLE Status
76	98	152	Walking	85%	Yes	Connected
80	97	160	Walking	84%	Yes	Connected
82	96	165	Running	83%	Yes	Connected
---	---	165	Standing	82%	No	Connected
78	98	170	Walking	82%	Yes	Connected
76	99	178	Walking	81%	Yes	Connected
72	98	180	Standing	80%	Yes	Connected
---	---	180	Standing	79%	No	Disconnected

RESULT ANALYSIS: -

The wearable health monitor accurately tracked vital signs and physical activity:

- **Heart Rate & SpO₂:** The MAX30102 reliably measured heart rate (72–82 BPM) and SpO₂ (96%–99%). "No Finger" detection worked well to prevent false readings.
- **Step Counting & Activity:** The BMA400 correctly recorded steps and classified activities like walking, running, and standing.
- **OLED & Haptic Feedback:** Clear real-time display with battery status. Haptic motor provided responsive user feedback.
- **BLE Transmission:** Sensor data was successfully broadcast via BLE in multiple states and received reliably by connected devices.
- **Power Management:** Battery voltage was monitored and displayed effectively; auto-off feature conserved energy.

CONCLUSION: -

The developed wearable device successfully integrates the ESP32, MAX30102, BMA400, and OLED display to continuously monitor heart rate, SpO₂, and physical activity. Real-time data logging and BLE transmission enable remote health tracking, offering a practical, cost-effective solution for personal and clinical use. The project demonstrates how embedded systems and IoT can be combined to improve health awareness, support early diagnosis, and promote preventive care through accessible technology.

FUTURE SCOPE: -

This wearable system can be further enhanced by:

- **Cloud Integration:** Enabling remote data storage and visualization through platforms like Firebase or AWS.
- **Additional Sensors:** Adding temperature, ECG, or blood pressure sensors for comprehensive health monitoring.
- **Machine Learning:** Implementing anomaly detection or health prediction algorithms.
- **Power Optimization:** Improving battery life with energy-efficient components and sleep modes.
- **GPS Integration:** Adding location-based tracking for outdoor fitness or emergency alerts.

These advancements would make the device more robust, user-friendly, and suitable for both consumer and medical-grade applications.